

UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECJ1023 Programming Technique II

Semester 2, 2025/2026

Group Project - Final Report

Project Title:

Food Order and Delivery Tracking System

Role	Full Name	Student ID
Group Leader	<i>Lim Li Jing</i>	<i>A25CS0048</i>
Member	<i>Lim Li Ning</i>	<i>A25CS0049</i>
Member	<i>Cheng Zhi Min</i>	<i>A25CS0050</i>
Member	<i>Ng Xuan Yee</i>	<i>A25CS0291</i>

Section: **03 / 04 / 06** (circle yours)

Lecturer: **Madam Lizawati Mi Yusuf**

Submission Date: **21 June 2026, 12:00 noon**

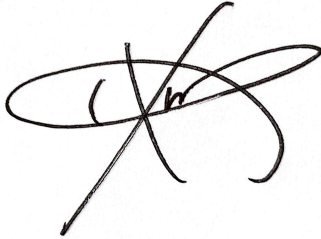
Group Member Contributions and Declaration

Each group member must complete their own declaration table below. All signatures must be **handwritten**. This section is assessed individually.

⚠ Failure to complete this section will affect your TW4 / PSA mark.

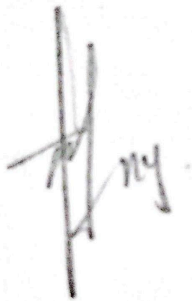
Member 1 — Declaration & Contribution			
Full Name	Cheng Zhi Min	Student ID	A25CS0050
Task(s) Assigned	- Section A - Identified and compiled the objectives and purpose of the system from Deliverable 1. - Provided explanation for the flowchart and UML class diagram, compiled and updated from Deliverable 2.		
Contribution Summary	- Created the MenuItem, FoodItem, and DrinkItem classes. - Completed all the class definitions of the three classes including private attributes and public members functions for each class. - Applied the concept of inheritance with MenuItem as base class and FoodItem/DrinkItem as derived classes. - Applied the concept of polymorphism by overriding display() function to print category or hot/cold output.		
Signature		Date	20/6/2026
Declaration: I confirm that the work described above is my own contribution to this group project and has not been plagiarised.			

Member 2 — Declaration & Contribution			
Full Name	Ng Xuan Yee	Student ID	A25CS0291
Task(s) Assigned	- Section B - Provided explanation for the implementation of OOP concepts used in the system. - Screenshot from the program code to further support the explanation,		
Contribution Summary	- Created the OrderItem and CustomerInfo classes. - Completed all the class definitions of the classes including private attributes and public members functions for each class. - Implemented 2 classes, one tracks a single order item and the other handles customer delivery information.		

Signature		Date	20/6/2026
Declaration: I confirm that the work described above is my own contribution to this group project and has not been plagiarised.			

Member 3 — Declaration & Contribution			
Full Name	Lim Li Jing	Student ID	A25CS0048
Task(s) Assigned	- Section A - Draw Flowchart and UML Class Diagram using draw.io based on Deliverable 2, updated version.		
Contribution Summary	- Created the Payment and Order classes. - Completed all the class definitions of the classes including private attributes and public members functions for each class. - Created payment processing with 3 payment methods and full order management including receipt generation.		
Signature		Date	20/6/2026
Declaration: I confirm that the work described above is my own contribution to this group project and has not been plagiarised.			

Member 4 — Declaration & Contribution			
Full Name	Lim Li Ning	Student ID	A25CS0049
Task(s) Assigned	- Section C - Completed the source code overview part by describing the structure and complexity of the code. - Filled in the user manual part by showing sample inputs and expected outputs for each feature.		
Contribution Summary	- Created the Feedback and FoodOrderingSystem classes. - Completed all the class definitions of the classes including private attributes and public members functions for each class.		

	- <i>Implemented the customer's feedback system and the main system class that integrates all components together.</i>		
Signature 		Date 20/6/2026	
Declaration: I confirm that the work described above is my own contribution to this group project and has not been plagiarised.			

Section A: Project Description and Design

Compile and update findings from Deliverable 1 and Deliverable 2. All diagrams must reflect the **final, complete system** — not an earlier draft.

A.1 Project Description

Describe the **objectives and purpose** of your system. Answer: What does the system do? What problem does it solve? Who is it for?

This is a console-based food ordering and delivery tracking application which makes use of C++ programming language to manage the whole process from ordering, payments, deliveries, and providing feedback on orders made.

The core capabilities are:

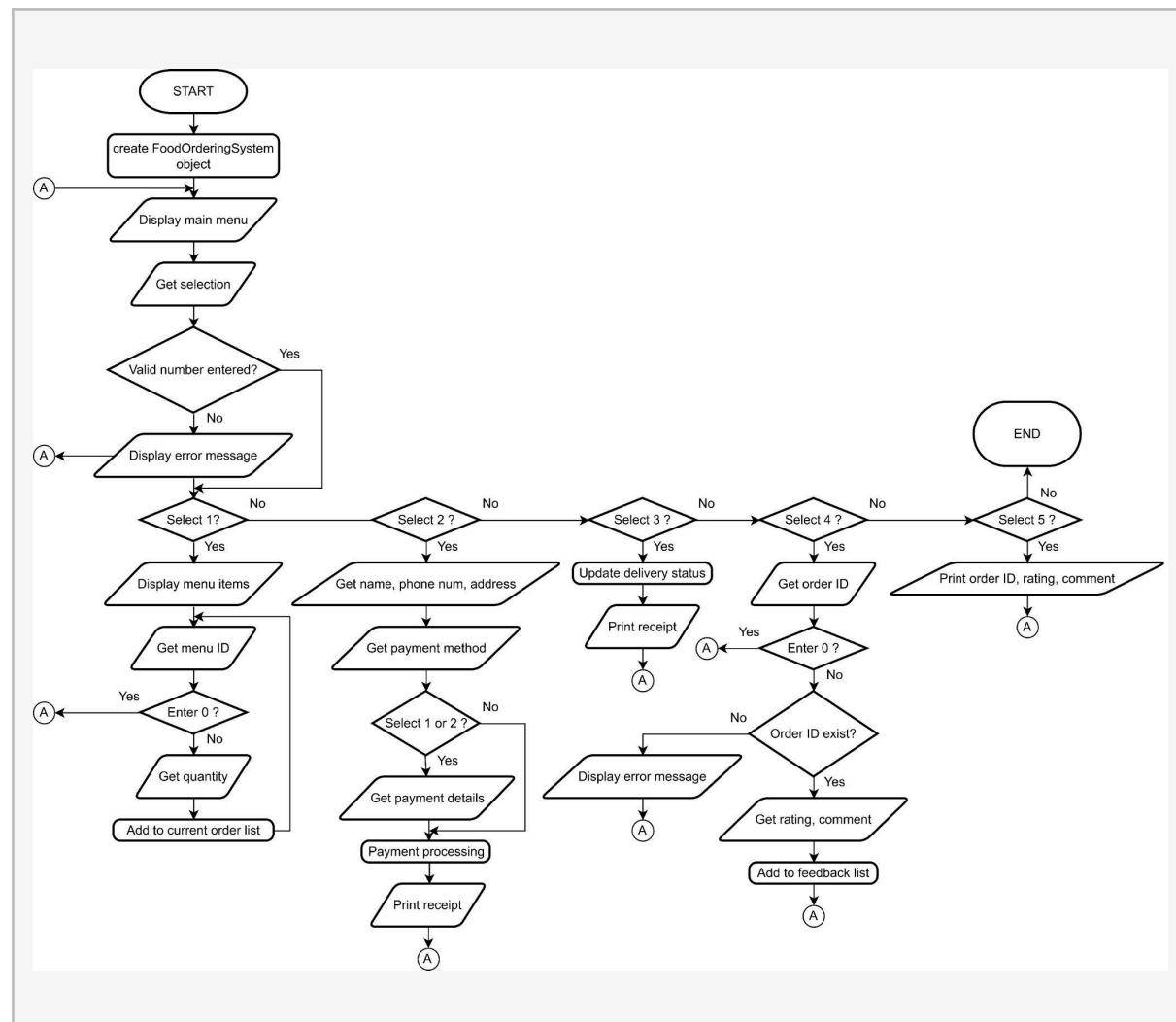
- 1. Menu & Ordering – In this system, customers can choose the order using its respective code along with quantity. The system merges all similar item's orders to make it an individual item.*
- 2. Checkout & Payment – Using this system, customers will be able to add their details such as name, phone number, and address during checkout. The system offers three types of payments including credit card/debit cards, e-wallets, and cash. Then, the system produces a receipt having a unique order ID.*
- 3. Live Tracking – The system enables customers to track their order status. The system rotates through four random delivery statuses: Preparing, Kitchen Cooking, On the Way, Arrived. This simulates real-time delivery progress.*
- 4. Feedback — The system enables customers to give a star rating ranging from 0 to 5 along with feedback about any order completed successfully. A single order can have only one rating at a time.*

The system minimizes errors while making the order, ensuring the entire process is streamlined. Without the system, manual ordering becomes more error-prone.

The intended customer for the system is everyone looking to place delivery orders without a mobile application. It can also cater to customers walking in or calling in.

A.2 System Workflow — Flowchart

Insert your **flowchart** below, then answer the explanation prompts. **A diagram without explanation will not receive full marks.**



Explanation

What does the flowchart show?

The program starts by displaying the main menu with 6 options. Each option is designed for different tasks in the food order and delivery tracking process (1 - Order Menu, 2 - Payment & Checkout, 3 - Live Tracking, 4 - Give Feedback, 5 - View All Feedbacks, 6 - Exit). If the user enters an invalid number, the program will display an error message and go back to the main menu. Lastly, the program ends if the user selects option 6, which the program will display a farewell message and terminate.

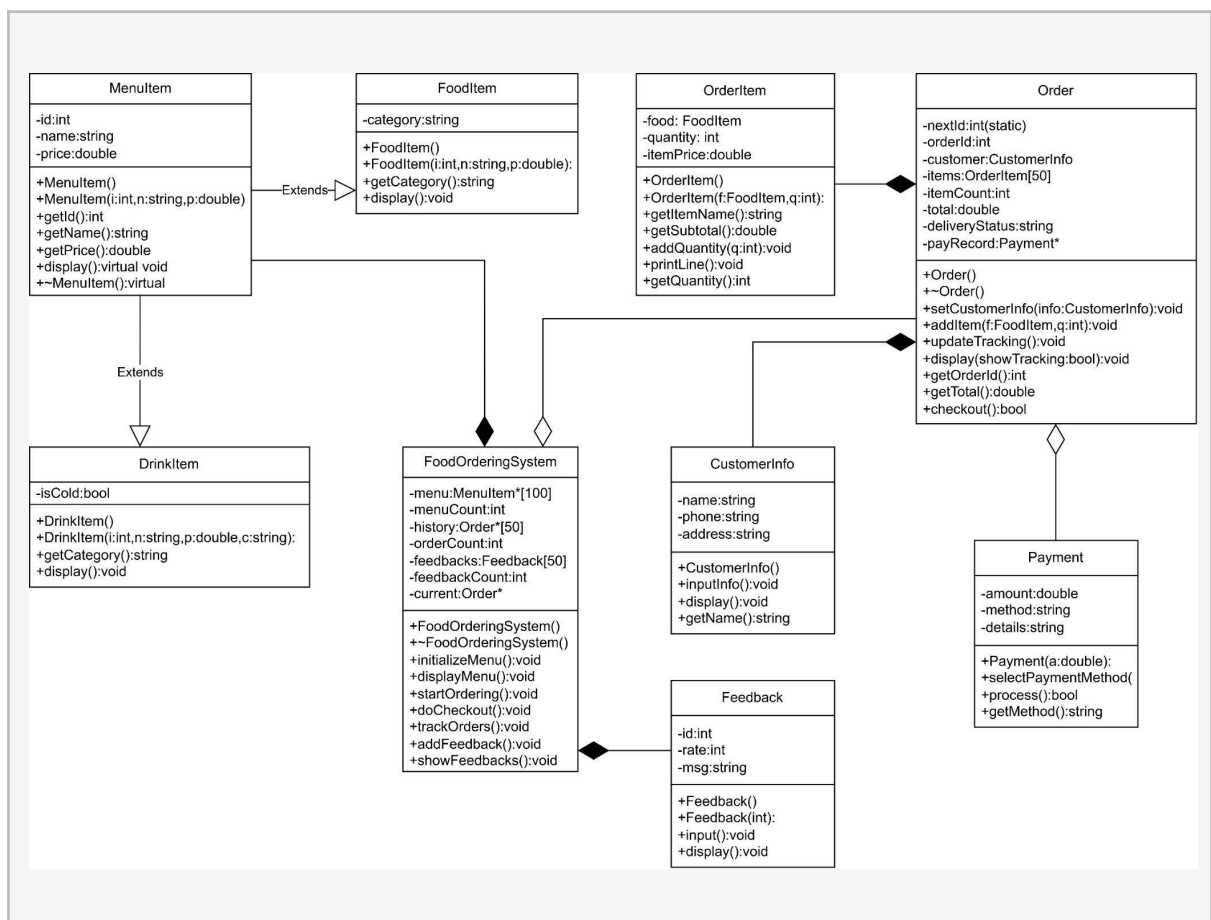
Start point

The starting point of the system is inside the `main()` function. Once the user runs the program, `main()` will create a `FoodOrderingSystem` object, which then triggers the class constructor to initialize the menu with 20 `FoodItem` and `DrinkItem` objects, set all counters to 0, reset the menu and order arrays, and seed the random number generator for later use in delivery status simulation. After the setup is completed, the `main()` function will enter into a continuous looping structure, which first displays the main menu, then prompts for user input (1-6) where each input will trigger different tasks, such as ordering, checkout, give feedback, tracking, or so on.

Key decision points	1. Main menu loop – program checks if the user enters a valid number (1-6). If not, the program will continuously display an error message and prompt the user until a valid number is entered. 2. Menu selection switch – once the user enters a valid number (1-6), the system routes to one of 6 paths (Order Menu, Checkout, Tracking, Give Feedback, View Feedback, Exit) and carries out its corresponding tasks. 3. Get menu ID loop – program checks if user enters 0 or not (stop ordering condition). If not, the program will continuously ask for menu ID and its quantity, add the order to the current list, until 0 is entered, program will return back to the main menu. 4. Payment method selection – If the user enters 1 or 2 (Card or E-Wallet), the program will prompt the user to enter payment details (Card number or Wallet ID) before payment processing. If not, the program assumes payment by cash and proceeds to payment processing. 5. Give Feedback feature – program checks if user enters an existing order ID, or 0 to cancel. If the order ID does not exist, the program displays an error message and returns back to the main menu. If the user enters 0 to the order ID, the program returns back to the main menu also.
End point	The system ends if the user selects option 6. Exit from the main menu. The system will return to the main menu each time a task is completed.

A.3 UML Class Diagram

Insert your **updated UML class diagram** below, then answer the explanation prompts. **A diagram without explanation will not receive full marks.**



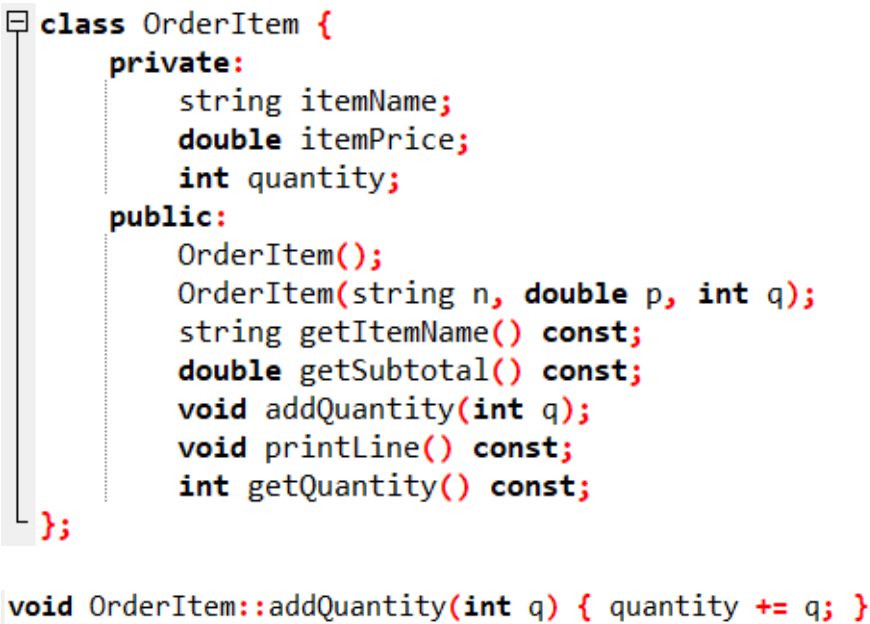
Explanation

Classes in your system	1. <i>MenuItem</i> – base class representing a generic menu entry with shared ID, name, and price. 2. <i>FoodItem</i> – represents a food item, extending <i>MenuItem</i> with a category label. 3. <i>DrinkItem</i> – represents a drink item, extending <i>MenuItem</i> with a hot/cold attribute. 4. <i>OrderItem</i> – represents one line within an order, storing name, price, and quantity. 5. <i>CustomerInfo</i> – stores and displays a customer's delivery details. 6. <i>Payment</i> – handles selecting a payment method and processing the payment amount. 7. <i>Order</i> – represents one complete customer order, managing items, total, status, and checkout. 8. <i>Feedback</i> – stores a customer's star rating and comments for a specific order. 9. <i>FoodOrderingSystem</i> – the main controller class coordinating the menu, orders, and feedback.
Association relationships	<i>Composition</i> 1. <i>Order</i> and <i>OrderItem</i> – <i>OrderItem</i> is stored as an array of objects inside <i>Order</i>, not as pointers, meaning they share the same memory lifetime. When an <i>Order</i> is destroyed, all its <i>OrderItems</i> are automatically destroyed with it. 2. <i>Order</i> and <i>CustomerInfo</i> – customer's delivery details are an intrinsic part of a specific transaction. If an <i>Order</i> is deleted, the <i>CustomerInfo</i> does not still exist as it is tied to that specific order's lifecycle. 3. <i>FoodOrderingSystem</i> and <i>Feedback</i> – feedback data is stored within the controller's memory space as a core part of its internal state. If the <i>FoodOrderingSystem</i> is deleted, the <i>Feedback</i> entries do not still exist. 4. <i>FoodOrderingSystem</i> and <i>MenuItem</i>* – <i>FoodOrderingSystem</i> creates and owns all <i>MenuItem</i> objects, which cannot exist independently outside of it. When <i>FoodOrderingSystem</i> is destroyed, it deletes them all, confirming full lifetime control. <i>Aggregation</i> 1. <i>FoodOrderingSystem</i> and <i>Order</i> – The system acts as a collection manager for historical records. If the <i>FoodOrderingSystem</i> is deleted, the <i>Order</i> objects stored in the history do still exist as independent entities until specifically deallocated 2. <i>Order</i> and <i>Payment</i> – The payment process is a separate functional entity responsible for transaction validation. If the <i>Order</i> is deleted, the <i>Payment</i> logic and its class structure do still exist independently and in some system designs, the payment record might be archived separately
Inheritance hierarchy	Parent class – <i>MenuItem</i> Derived classes – <i>FoodItem</i>, <i>DrinkItem</i> The <i>FoodItem</i> class and <i>DrinkItem</i> class inherit 3 private attributes from <i>MenuItem</i> class – id, name, price.
Polymorphism	The <i>display()</i> method is overridden in <i>FoodItem</i> class and <i>DrinkItem</i> class. <i>FoodItem::display()</i> extends <i>MenuItem::display()</i> by displaying the private attribute of <i>FoodItem</i> – category (main, snack, or dessert), which determines the category of the food. <i>DrinkItem::display()</i> extends <i>MenuItem::display()</i> by displaying the private attribute of <i>DrinkItem</i> – <i>isCold</i> boolean (true – cold, false – hot), which determines whether the drink is hot or cold.

Section B: Implementation of OOP Concepts

For each concept, you must: (1) define it in your own words, (2) explain specifically how it is used in your system, and (3) reference the source code. **Do not copy definitions from the internet.**

B.1 Encapsulation

Encapsulation	
What it is	<i>Encapsulation is to restrict direct access of an object's data by declaring it private, and only public methods can access the data.</i>
How it is used in your system	<i>All classes have encapsulation implemented into them. For example, class OrderItem uses three private variables: itemName, itemPrice, and quantity. This means that nothing outside of the OrderItem can use them in any shape or form because they are declared private. However, there is an easy way to access those variables – by using certain public functions: getItemName(), getSubtotal(), getQuantity() and addQuantity(). For example, there is no chance to accidentally make the variable quantity below because the only method to do so is to call addQuantity() passing in positive numbers. The same pattern applies across CustomerInfo, Payment, Feedback, and MenuItem — all hide their fields and expose only what other classes genuinely need.</i>
Class / File	OrderItem.h / OrderItem.cpp
Code Snippet or Screenshot	 <pre>class OrderItem { private: string itemName; double itemPrice; int quantity; public: OrderItem(); OrderItem(string n, double p, int q); string getItemName() const; double getSubtotal() const; void addQuantity(int q); void printLine() const; int getQuantity() const; }; void OrderItem::addQuantity(int q) { quantity += q; }</pre>

B.2 Composition

Composition	
What it is	<i>Composition is a "has-a" relationship where one class owns another class as a data member, and the owned object's lifetime is fully controlled by the owning class.</i>
How it is used in your system	<i>The most fitting example in this regard would be the Order class, which consists of the classes CustomerInfo and OrderItem. Both classes are used to be members of value type for the Order class. When an Order object is constructed, both the</i>

	<i>CustomerInfo</i> object along with the <code>MAX_ORDER_ITEMS</code> number of <i>OrderItems</i> are constructed. Similarly, while destroying an instance of the <i>Order</i> class, all of them are destroyed as well. The reason being, these two classes have no significance without the <i>Order</i> that contains them. <i>FoodOrderingSystem</i> also illustrates composition through its array of <i>Feedback</i> objects (<code>feedbacks[MAX_FEEDBACKS]</code>), so this means that the class also follows the same pattern.
Class / File	<code>Order.h / Order.cpp</code>
Code Snippet or Screenshot	<pre> class Order { private: static int nextId; int orderId; CustomerInfo customer; OrderItem items[MAX_ORDER_ITEMS]; int itemCount; double total; string deliveryStatus; Payment* payRecord; public: void Order::setCustomerInfo(CustomerInfo info) { customer = info; } items[itemCount++] = OrderItem(name, price, q); items[i].addQuantity(q); items[i].printLine(); </pre>

B.3 Aggregation

Aggregation	
What it is	Aggregation is a "has-a" relationship where one class references or uses another class, but the referenced object has its own independent lifetime and can exist without the owning class.
How it is used in your system	The best case would be the aggregation of <i>MenuItem</i> instances as <i>FoodItems</i> or <i>DrinkItems</i> through the <i>FoodOrderingSystem</i> class. In this way, initialization is done just once in <code>initializeMenu()</code> and they remain in the menu pointer array forever after that point. While placing the order, <i>FoodOrderingSystem</i> provides information about the name and cost of the menu item, but no change is made in the actual <i>MenuItem</i> objects because these objects are independent and live in the menu array without belonging to any order. This is so because these menu items have been created separately as individual instances which are used in every order placed by the customer. The menu pointer array in <i>FoodOrderingSystem</i> holds references to these items, and only the destructor of <i>FoodOrderingSystem</i> removes these items.
Class / File	<code>FoodOrderingSystem.h / FoodOrderingSystem.cpp</code>

Code Snippet or Screenshot

```
class FoodOrderingSystem {
private:
    MenuItem* menu[MAX_MENU * 2];
    int menuCount;
    Order* history[MAX_ORDERS];
    int orderCount;
    Feedback feedbacks[MAX_FEEDBACKS];
    int feedbackCount;
    Order* current;
public:

void FoodOrderingSystem::initializeMenu() {
    menu[menuCount++] = new FoodItem(1, "Beef Burger Special", 12.00, "Main");
    menu[menuCount++] = new FoodItem(2, "Char Kuey Teow", 7.00, "Main");
    menu[menuCount++] = new FoodItem(3, "Chicken Pizza (R)", 12.00, "Main");
    menu[menuCount++] = new FoodItem(4, "Chicken Rice", 9.00, "Main");
    menu[menuCount++] = new FoodItem(5, "Nasi Lemak Chicken", 7.50, "Main");
    menu[menuCount++] = new FoodItem(6, "Penang Hokkien Mee", 7.00, "Main");
    menu[menuCount++] = new FoodItem(7, "Penang Laksa", 7.00, "Main");
    menu[menuCount++] = new FoodItem(8, "Wonton Noodle Soup", 8.50, "Main");
    menu[menuCount++] = new FoodItem(9, "Cheesy Fries", 5.50, "Snack");
    menu[menuCount++] = new FoodItem(10, "Curry Puff Sardin (2 pcs)", 4.00, "Snack");
    menu[menuCount++] = new FoodItem(11, "Fried Chicken (2 pcs)", 7.00, "Snack");
    menu[menuCount++] = new FoodItem(12, "Roti Canai", 3.50, "Snack");
    menu[menuCount++] = new FoodItem(13, "Tosai", 4.50, "Snack");
    menu[menuCount++] = new FoodItem(14, "Apam Balik (2 pcs)", 4.00, "Dessert");
    menu[menuCount++] = new FoodItem(15, "Burnt Cheese Cake (1 pcs)", 7.00, "Dessert");
    menu[menuCount++] = new FoodItem(16, "Sundae Cone", 4.00, "Dessert");
    menu[menuCount++] = new DrinkItem(17, "Cendol", 4.00, true);
    menu[menuCount++] = new DrinkItem(18, "Iced Lemon Tea", 4.00, true);
    menu[menuCount++] = new DrinkItem(19, "Teh Tarik Ais", 3.00, true);
    menu[menuCount++] = new DrinkItem(20, "Teh Tarik Panas", 3.00, false);
}

current->addOrMergeItem(menu[i]->getName(), menu[i]->getPrice(), qty);

FoodOrderingSystem::~FoodOrderingSystem() {
    for(int i = 0; i < orderCount; i++)
        delete history[i];
    for(int i = 0; i < menuCount; i++)
        delete menu[i];
    if(current)
        delete current;
}
```

B.4 Inheritance

Inheritance	
What it is	<i>Inheritance is a mechanism where a derived class acquires the attributes and behaviours of a base class, extending or specialising them without rewriting shared logic.</i>
How it is used in your system	<i>MenuItem acts as the base class, whereas FoodItem and DrinkItem classes extend this base class. Common attributes for a menu item, which are id, name, and price along with their common getter methods getId(), getName(), and getPrice() have been included in the base class. To avoid duplication of these common attributes into the derived class definitions, only declare them once in the base class and will be inherited by the derived classes. Furthermore, there will be</i>

	one more property in each of the derived classes as compared to those of the base class. For instance, the additional property of the <i>FoodItem</i> is <i>category</i> while that of <i>DrinkItem</i> is <i>isCold</i>. Moreover, there will also be an overriding of <i>display()</i> method for displaying the special attribute of each of the derived classes, while the base <i>MenuItem::display()</i> will simply act as a backup display. Constructors of the derived class utilize the initializer list syntax (<i>: MenuItem(i, n, p)</i>) to explicitly call the base constructor. This can make sure that the shared fields are set through the base class rather than duplicated.
Class / File	<i>MenuItem.h</i> / <i>MenuItem.cpp</i> (base) <i>FoodItem.h</i> / <i>FoodItem.cpp</i> (derived) <i>DrinkItem.h</i> / <i>DrinkItem.cpp</i> (derived)
Code Snippet or Screenshot	<pre> class MenuItem { protected: int id; string name; double price; public: MenuItem(); MenuItem(int i, string n, double p); int getId() const; string getName() const; double getPrice() const; virtual void display() const; virtual ~MenuItem() {} }; class FoodItem : public MenuItem { private: string category; public: FoodItem(); FoodItem(int i, string n, double p, string c); string getCategory() const; void display() const; }; class DrinkItem : public MenuItem { private: bool isCold; public: DrinkItem(); DrinkItem(int i, string n, double p, bool cold); bool getIsCold() const; void display() const; }; FoodItem::FoodItem(int i, string n, double p, string c) : MenuItem(i, n, p) { category = c; } </pre>

	<pre> void FoodItem::display() const { cout << " " << setw(2) << id << ". " << left << setw(26) << name << "RM" << fixed << setprecision(2) << right << setw(6) << price << " [Food - " << category << "]" << endl; } DrinkItem::DrinkItem(int i, string n, double p, bool cold) : MenuItem(i, n, p) { isCold = cold; } void DrinkItem::display() const { cout << " " << setw(2) << id << ". " << left << setw(26) << name << "RM" << fixed << setprecision(2) << right << setw(6) << price << " [Drink - " << (isCold ? "Cold" : "Hot") << "]" << endl; } </pre>
--	---

B.5 Polymorphism

Polymorphism	
What it is	Polymorphism is the ability for a base class pointer or reference to call the correct overridden method of a derived class at runtime, without the caller needing to know the actual type of the object.
How it is used in your system	FoodOrderingSystem stores all menu items in a single array of MenuItem* pointers — a mix of FoodItem and DrinkItem objects. When displayMenu() loops through that array and calls menu[i]->display(), it does not check whether each entry is a food or a drink. Depending upon whether it is a pointer to a FoodItem object or a pointer to a DrinkItem object, either FoodItem::display() or DrinkItem::display(), respectively, will be invoked automatically by the computer when the program runs. This form of polymorphism is called run-time polymorphism since the computer determines which of the two member functions to invoke when the program executes. This is possible because of the keyword virtual placed before the function display(). Without this word, all calls to display() will resolve to MenuItem::display(). There would then be no trace left of what category the item belongs to and whether it is hot or cold. The virtual ~MenuItem() destructor follows the same principle — ensuring that when a MenuItem* is deleted, the correct derived destructor runs, preventing resource leaks.
Class / File	MenuItem.h (virtual declaration) FoodItem.h / FoodItem.cpp (override) DrinkItem.h / DrinkItem.cpp (override) FoodOrderingSystem.h / FoodOrderingSystem.cpp (polymorphic usage)
Code Snippet or Screenshot	<pre> virtual void display() const; virtual ~MenuItem() {} class FoodOrderingSystem { private: MenuItem* menu[MAX_MENU * 2]; } </pre>

	<pre> void FoodOrderingSystem::initializeMenu() { menu[menuCount++] = new FoodItem(1, "Beef Burger Special", 12.00, "Main"); menu[menuCount++] = new FoodItem(2, "Char Kuey Teow", 7.00, "Main"); menu[menuCount++] = new FoodItem(3, "Chicken Pizza (R)", 12.00, "Main"); menu[menuCount++] = new FoodItem(4, "Chicken Rice", 9.00, "Main"); menu[menuCount++] = new FoodItem(5, "Nasi Lemak Chicken", 7.50, "Main"); menu[menuCount++] = new FoodItem(6, "Penang Hokkien Mee", 7.00, "Main"); menu[menuCount++] = new FoodItem(7, "Penang Laksa", 7.00, "Main"); menu[menuCount++] = new FoodItem(8, "Wonton Noodle Soup", 8.50, "Main"); menu[menuCount++] = new FoodItem(9, "Cheesy Fries", 5.50, "Snack"); menu[menuCount++] = new FoodItem(10, "Curry Puff Sardin (2 pcs)", 4.00, "Snack"); menu[menuCount++] = new FoodItem(11, "Fried Chicken (2 pcs)", 7.00, "Snack"); menu[menuCount++] = new FoodItem(12, "Roti Canai", 3.50, "Snack"); menu[menuCount++] = new FoodItem(13, "Tosai", 4.50, "Snack"); menu[menuCount++] = new FoodItem(14, "Apam Balik (2 pcs)", 4.00, "Dessert"); menu[menuCount++] = new FoodItem(15, "Burnt Cheese Cake (1 pcs)", 7.00, "Dessert"); menu[menuCount++] = new FoodItem(16, "Sundae Cone", 4.00, "Dessert"); menu[menuCount++] = new DrinkItem(17, "Cendol", 4.00, true); menu[menuCount++] = new DrinkItem(18, "Iced Lemon Tea", 4.00, true); menu[menuCount++] = new DrinkItem(19, "Teh Tarik Ais", 3.00, true); menu[menuCount++] = new DrinkItem(20, "Teh Tarik Panas", 3.00, false); } void FoodOrderingSystem::displayMenu() { cout << "\n===== MENU =====" << endl; for(int i = 0; i < menuCount; i++) menu[i]->display(); cout << endl; } </pre>
--	--

B.6 Array of Objects

Array of Objects	
What it is	Arrays of objects mean that there is an array of a definite size that holds the members or elements of a particular kind of object or belongs to a particular class.
How it is used in your system	Objects arrays are used in the following instances: 1. Order holds an array of objects of type OrderItem — items[MAX_ORDER_ITEMS]. Each of these objects is already created completely and contains all details of items ordered (name, price, quantity). They are referenced through the itemCount and iterated over whenever orders are printed and processed. In case two or more items are of the same name in the same order, then the search within the array will take place based on names and call addQuantity() rather than adding a new slot. 2. FoodOrderingSystem holds an array of Feedback value objects — feedbacks[MAX_FEEDBACKS]. What this implies is that every element in the array is the feedback given by a certain customer concerning their experience with the services provided. The Feedback object is indexed and added based on the feedbackCount, checked to avoid duplication of ratings for each order, and iterated in showFeedbacks() to print all reviews. Both arrays store objects by value (not pointers), meaning each element is a fully self-contained object living directly inside the array's memory block.
Class / File	Order.h / Order.cpp FoodOrderingSystem.h / FoodOrderingSystem.cpp

```

const int MAX_ORDER_ITEMS = 50;

class Order {
private:
    static int nextId;
    int orderId;
    CustomerInfo customer;
    OrderItem items[MAX_ORDER_ITEMS];

bool Order::addOrMergeItem(string name, double price, int q) {
    // Check if item already exists
    for (int i = 0; i < itemCount; i++) {
        if (items[i].getItemName() == name) {
            items[i].addQuantity(q);
            total += price * q;
            return true; // merged
        }
    }
    // Not found – add new entry
    if (itemCount < MAX_ORDER_ITEMS) {
        items[itemCount++] = OrderItem(name, price, q);
        total += price * q;
        return true;
    }
    return false; // order full
}

for(int i = 0; i < itemCount; i++)
    items[i].printLine();

const int MAX_MENU = 50;
const int MAX_ORDERS = 50;
const int MAX_FEEDBACKS = 50;

class FoodOrderingSystem {
private:
    MenuItem* menu[MAX_MENU * 2];
    int menuCount;
    Order* history[MAX_ORDERS];
    int orderCount;
    Feedback feedbacks[MAX_FEEDBACKS];

    // Check if already rated
    for (int i = 0; i < feedbackCount; i++) {
        if (feedbacks[i].getId() == oid) {
            cout << "[!] You have already rated order #" << oid << "." << endl;
            return;
        }
    }
    Feedback fb(oid);
    fb.input();
    feedbacks[feedbackCount++] = fb;

```


Section C: User Manual and System Demonstration

This section has two parts:

Part 1 - Source Code Overview: Describe the structure and complexity of your code.

Part 2 - User Manual: Demonstrate your program by showing sample inputs and expected outputs for each feature.

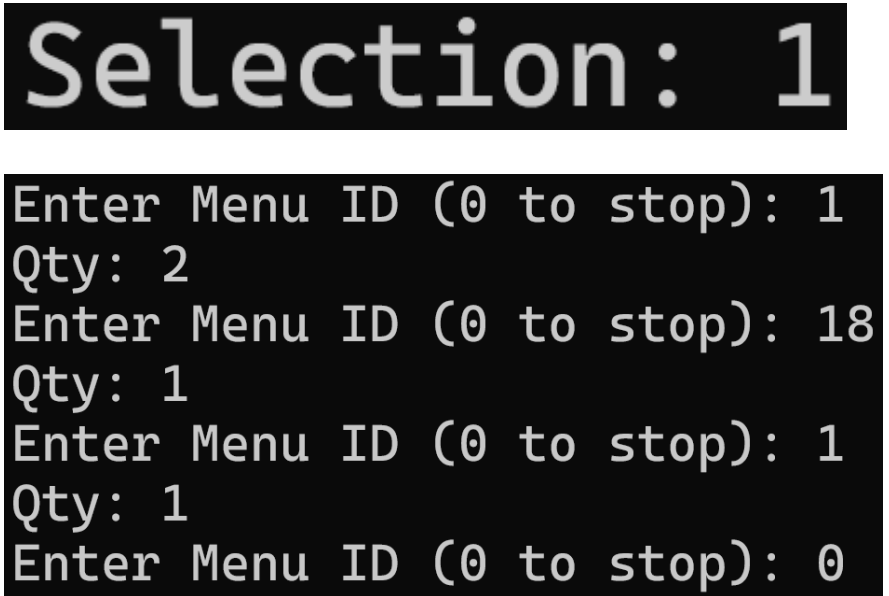
Part 1: Source Code Overview

Source Code Overview	
Number of classes	9 classes: MenuItem, FoodItem, DrinkItem, Order, OrderItem, CustomerInfo, Payment, Feedback, FoodOrderingSystem
System complexity	<i>There are four main aspects of Object-oriented Programming, which need to be taken into account during the creation of the design of the system. The concepts related to Object-oriented programming such as composition, aggregation, inheritance and polymorphism are taken into account while designing the system. There are five main functions of the system which are exploring the menu, ordering, payment, delivery and providing feedback. Some examples of association classes that are being considered during the proposed development of the system are inheritance, composition, aggregation, and destructor based dynamic memory management.</i>
Code organisation	<i>The classes are organized into two files. One file is considered as header file whereas the other file as the implementation file. This is according to the common organization structure when coding using C++ language. The header files have header guards such as #ifndef, #define and #endif in every file. This ensures that there is no inclusion problem in the project. This ensures that the inclusion problem will not be an issue with the program. Since both the class and the header file share the same name, it is extremely easy to understand. FoodOrderingSystem sits at the top of the hierarchy, pulling in all other headers. main.cpp only includes FoodOrderingSystem.h.</i>
How the system works as a whole	<i>In main.cpp, there is an object created of type FoodOrderingSystem and it uses a loop to show a main menu with 6 options to the user. All user interaction is routed through this one controller object.</i> <i>The FoodOrderingSystem constructor invokes initializeMenu(), resulting in allocation on the heap of 20 MenuItem* items of which 16 FoodItems, and 4 DrinkItems held in the menu[] pointer array. Thereafter, the sequence of events follow the process of ordering a meal:</i> <i>1. Ordering -> startOrdering() gets triggered by the creation of an Order object, which helps in selecting items through the use of ID. For every item selected, the name and price of such item are added to a new instance of the OrderItem from the related MenuItem object to the internal items[] array of Order object. Duplicate selections are merged rather than duplicated.</i> <i>2. Checkout -> doCheckout() leads to a CustomerInfo input which will be contained within the Order. Next, the system calls the checkout() method in the Order class to create a Payment object, processes the payment, and prints a receipt. The system then archives the completed Order into the history[] array.</i> <i>3. Tracking -> trackOrders() loops through history[] array and invokes updateTracking() method to update randomly the string depicting the delivery status of an order.</i> <i>4. Feedback -> addfeedback() enables the user to pick any order ID that has already been created and creates an object for it. However, it will be added to the feedbacks[] array only when there has been no other feedback submitted by the user before on that particular order ID.</i>

Throughout the entire session, the destructor of the class `FoodOrderingSystem` makes sure that all the heap-allocated objects of the types `MenuItem` and `Order` are deleted on exit.

Part 2: User Manual

For each feature or task in your system, describe how it works and show sample inputs and outputs. Use screenshots where possible.

C.1 Feature / Task 1: Order Menu	
Description	<i>One of the best things about this feature is that the customer will have an opportunity to get anything that features in the menu. This happens during the initial stage of the process, forming the basis for everything else, like checking out, feedback, and so on.</i>
How to use it	1. From the main menu, select 1. Order Menu. 2. The entire menu will pop up on screen; the items will be organized by default based on their IDs; 16 food items (Main, Snack, and Dessert) and 4 drinks (Hot/Cold drinks). 3. Enter a Menu ID to add that item, or 0 to stop ordering. 4. If the customer inputs the right menu ID, then it will require the quantity of the items entered in the order. Putting any negative integer will give an error message. 5. Steps 3 and 4 could be done multiple times until customers wish to stop ordering. Customers may enter the same menu ID multiple times, resulting in increasing its quantity. 6. Enter 0 at the ID prompt to finish and return to the main menu.
Sample Input	 <pre>Selection: 1 Enter Menu ID (0 to stop): 1 Qty: 2 Enter Menu ID (0 to stop): 18 Qty: 1 Enter Menu ID (0 to stop): 1 Qty: 1 Enter Menu ID (0 to stop): 0</pre>

Expected Output

```
--- FOODIE DELIVERY ---
1. Order Menu
2. Payment & Checkout
3. Live Tracking
4. Give Feedback
5. View All Feedbacks
6. Exit

Selection: 1

===== MENU =====
 1. Beef Burger Special      RM 12.00 [Food - Main]
 2. Char Kuey Teow          RM  7.00 [Food - Main]
 3. Chicken Pizza (R)       RM 12.00 [Food - Main]
 4. Chicken Rice            RM  9.00 [Food - Main]
 5. Nasi Lemak Chicken      RM  7.50 [Food - Main]
 6. Penang Hokkien Mee     RM  7.00 [Food - Main]
 7. Penang Laksa            RM  7.00 [Food - Main]
 8. Wonton Noodle Soup     RM  8.50 [Food - Main]
 9. Cheesy Fries            RM  5.50 [Food - Snack]
10. Curry Puff Sardin (2 pcs) RM  4.00 [Food - Snack]
11. Fried Chicken (2 pcs)   RM  7.00 [Food - Snack]
12. Roti Canai              RM  3.50 [Food - Snack]
13. Tosai                   RM  4.50 [Food - Snack]
14. Apam Balik (2 pcs)     RM  4.00 [Food - Dessert]
15. Burnt Cheese Cake (1 pcs) RM  7.00 [Food - Dessert]
16. Sundae Cone             RM  4.00 [Food - Dessert]
17. Cendol                  RM  4.00 [Drink - Cold]
18. Iced Lemon Tea          RM  4.00 [Drink - Cold]
19. Teh Tarik Ais           RM  3.00 [Drink - Cold]
20. Teh Tarik Panas         RM  3.00 [Drink - Hot]

Enter Menu ID (0 to stop): 1
Qty: 2
Enter Menu ID (0 to stop): 18
Qty: 1
Enter Menu ID (0 to stop): 1
Qty: 1
Enter Menu ID (0 to stop): 0

--- FOODIE DELIVERY ---
1. Order Menu
2. Payment & Checkout
3. Live Tracking
4. Give Feedback
5. View All Feedbacks
6. Exit

Selection: |
```

C.2 Feature / Task 2: Payment and Checkout

Description	<i>This feature makes it simpler for all the orders to be delivered since the current order that is being served requires some delivery information, which will help it be completed successfully via the process of making the payment and creating the receipt. It converts the in-progress order built in the "Order Menu" feature into a confirmed, archived order with a permanent Order ID.</i>
How to use it	1. From the main menu, select 2. Payment & Checkout. 2. Where there is no order made before, a message of error will appear followed by redirection to the main menu page. 3. Enter delivery details when prompted: Receiver Name, Phone Number, and Address. 4. Choose a payment method: 1 for Card, 2 for E-Wallet, or 3 for Cash. 5. If Card or E-Wallet is selected, enter the corresponding Card No. or Wallet ID. Cash requires no extra input. 6. The process will be successful, leading to generation of a receipt with full details of the order such as Order ID and total cost.

Sample Input

Selection: 2

Receiver Name: Abu Bakar
Phone Number : 0123456789
Address : UTM Johor Bahru

Select Payment: 1
Card No: 1111222233334444

Expected Output

```
--- FOODIE DELIVERY ---
1. Order Menu
2. Payment & Checkout
3. Live Tracking
4. Give Feedback
5. View All Feedbacks
6. Exit

Selection: 2

===== DELIVERY INFORMATION =====
Receiver Name: Abu Bakar
Phone Number : 0123456789
Address      : UTM Johor Bahru

1. Card | 2. E-Wallet | 3. Cash
Select Payment: 1
Card No: 1111222233334444

>> Paying RM40.00 via Card... Done!

[Success] Receipt Generated:

-----
ORDER ID: #1001
STATUS: [Preparing]
-----

Customer: Abu Bakar
Contact : 0123456789
Address : UTM Johor Bahru

Beef Burger Special      x 3 = RM 36.00
Iced Lemon Tea           x 1 = RM  4.00

TOTAL: RM 40.00
-----

--- FOODIE DELIVERY ---
1. Order Menu
2. Payment & Checkout
3. Live Tracking
4. Give Feedback
5. View All Feedbacks
6. Exit

Selection: |
```

C.3 Feature / Task 3: Live Tracking

Description

This function allows users to experience live tracking of their deliveries for all the orders that have already been made. It automatically randomizes the delivery status for each previously ordered product whenever accessed, thereby providing an actual delivery application experience.

How to use it

1. From the main menu, select 3. Live Tracking.
2. If no orders have been checked out yet, the system displays an error and returns to the main menu.
3. The system automatically refreshes the delivery status of every past order — no further input is required.
4. All orders are displayed with their current status, delivery details, items, and total.

Sample Input

Selection: 3

Expected Output	<pre> --- FOODIE DELIVERY --- 1. Order Menu 2. Payment & Checkout 3. Live Tracking 4. Give Feedback 5. View All Feedbacks 6. Exit Selection: 3 ===== LIVE TRACKING ===== ----- ORDER ID: #1001 STATUS: [On the way] ----- Customer: Abu Bakar Contact : 0123456789 Address : UTM Johor Bahru Beef Burger Special x 3 = RM 36.00 Iced Lemon Tea x 1 = RM 4.00 TOTAL: RM 40.00 ----- --- FOODIE DELIVERY --- 1. Order Menu 2. Payment & Checkout 3. Live Tracking 4. Give Feedback 5. View All Feedbacks 6. Exit Selection: </pre>
-----------------	---

C.4 Feature / Task 4: Give Feedback	
Description	<i>This makes sure that the customer is able to rate the particular order by means of the five-star rating system from zero to five stars. In addition, comments are possible. This helps prevent any accidental duplication of ratings.</i>
How to use it	1. From the main menu, select 4. Give Feedback. 2. An error message is displayed in case the order has not been checked out prior to this choice. 3. A list will be shown with all past orders having Order IDs tagged as either [Rated] or [Not Rated].

	4. Enter the Order ID to rate, or 0 to cancel. 5. In case of a problem regarding the Order ID provided by the customer or already rated, an error message is generated. 6. If valid, enter a Rating from 0–5 and a Comment. 7. The system confirms the feedback was submitted successfully.
Sample Input	<pre> Selection: 4 Enter Order ID to rate (0 to cancel): 1001 Rating (0–5): 5 Comment: Very Good </pre>
Expected Output	<pre> --- FOODIE DELIVERY --- 1. Order Menu 2. Payment & Checkout 3. Live Tracking 4. Give Feedback 5. View All Feedbacks 6. Exit Selection: 4 ===== YOUR ORDERS ===== Order #1001 [Not Rated] Enter Order ID to rate (0 to cancel): 1001 Rating (0–5): 5 Comment: Very Good [Success] Feedback submitted for Order #1001! --- FOODIE DELIVERY --- 1. Order Menu 2. Payment & Checkout 3. Live Tracking 4. Give Feedback 5. View All Feedbacks 6. Exit Selection: </pre>

C.5 Feature / Task 5: View All Feedbacks

Description	All the feedback given up to now for the items purchased are shown by this module. In other words, this will provide an overall idea of the satisfaction level of customers as all the information will be provided at once.
How to use it	1. From the main menu, select 5. View All Feedbacks.

	- If no feedback is provided yet, an alert message saying so will be displayed. - All provided feedback will otherwise be displayed in order of receipt along with Order ID, star rating and comments.
Sample Input	<pre>Selection: 5</pre>
Expected Output	<pre> --- FOODIE DELIVERY --- 1. Order Menu 2. Payment & Checkout 3. Live Tracking 4. Give Feedback 5. View All Feedbacks 6. Exit Selection: 5 Order #1001 -> 5 stars: Very Good --- FOODIE DELIVERY --- 1. Order Menu 2. Payment & Checkout 3. Live Tracking 4. Give Feedback 5. View All Feedbacks 6. Exit Selection: </pre>